

Clavis-IAM

A Lightweight, Multi-Tenant OAuth 2.0 & OpenID Connect Identity Provider built with Spring Boot 3 & Security 6.

1. Project Overview & Scope ("Keycloak-Lite" Blueprint)

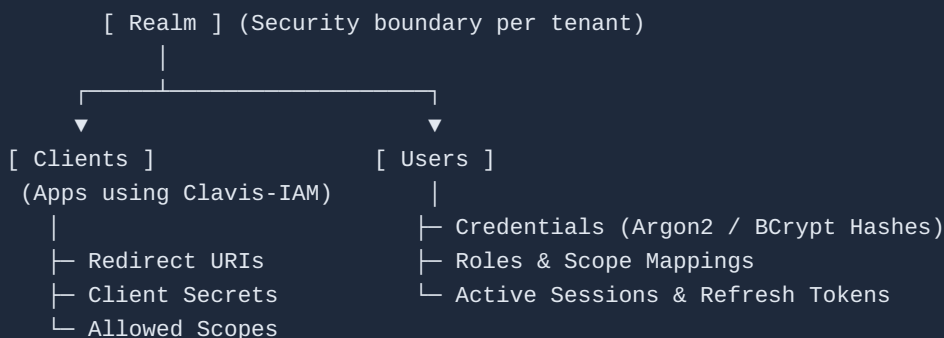
Clavis-IAM (derived from the Latin *clavis* meaning "key" or "lock") is a lightweight, cloud-native Identity and Access Management (IAM) server designed to deliver modern OAuth 2.0 and OpenID Connect (OIDC) capabilities without the bloat of legacy Java EE ecosystem tooling.

Core Engineering Objectives

- **OIDC Provider & Authorization Server:** Implements Authorization Code Flow with PKCE (RFC 7636) for modern web and mobile applications.
- **Stateless Token Minting & Validation:** Issues cryptographically signed RSA/EC Access Tokens, ID Tokens, and manages Refresh Token lifecycles.
- **Multi-Tenancy ("Realms"):** Dynamic logical isolation boundaries separating users, credentials, clients, and role-based access control policies per tenant.
- **Embedded User & Consent UI:** Thymeleaf and Tailwind CSS powered server-rendered interfaces for Login, Registration, MFA verification, and OIDC Consent screens.
- **Administrative REST API:** Comprehensive management endpoints for programmatically configuring realms, provisioning client applications, and managing identity life cycles.
- **Distributed Session Management:** Redis-backed dynamic token revocation lists (JTI blacklisting) and real-time active user session tracking.

2. Core Concepts & Architectural Blueprint

Understanding the domain model of modern identity providers allows for a clean separation of concerns in database design and request processing pipelines.



Key IAM Concepts Implemented

Concept	Description & Implementation Detail
Realm	An isolated security domain. Configured under dynamic paths like <code>/auth/realms/{realm-name}/...</code> allowing complete multi-tenant separation.
Client	An application asking Clavis-IAM to authenticate a user (e.g., Single Page App, Mobile App, or Microservice).
Grant Types	Focuses strictly on <code>authorization_code</code> with PKCE (for end users) and <code>client_credentials</code> (for machine-to-machine interactions).
Scopes & Claims	Standard OIDC scopes (<code>openid</code> , <code>profile</code> , <code>email</code>) mapped dynamically into signed JWT claims.

3. High-Level Class & Domain Design

Entity Relationship Model



Core Service & Security Components

- `JwtTokenProvider` : Responsible for RSA/EC key pair generation, JWS signing using `NimbusJwtEncoder` , and JWK formatting.
- `JwksController` : Exposes `/well-known/jwks.json` per realm, enabling downstream resource servers to validate tokens statelessly.
- `CustomUserDetailsService` : Resolves identities dynamically by combining target `Realm` context with input credentials.
- `PkceValidationFilter` : Custom security filter verifying `code_challenge` and `code_verifier` during token exchange.
- `TokenBlacklistService` : High-performance Redis service tracking revoked token identifiers (`jti`) for instant revocation.

- `AdminControllers` : REST endpoints managing realms, clients, and user resources under `/api/v1/admin/...`.

4. Implementation Roadmap

Phase 1: OAuth2 / OIDC Core Engine

- Configure Spring Boot 3 with `spring-boot-starter-oauth2-authorization-server`.
- Set up dynamic key rotation and RSA public/private key store management.
- Implement standard discovery metadata endpoint: `/well-known/openid-configuration`.
- Establish PostgreSQL domain schema using JPA / Liquibase migrations.

Phase 2: Multi-Tenancy & Custom UI

- Implement dynamic multi-tenant URL routing: `/auth/realms/{realm}/protocol/openid-connect/auth`.
- Build custom responsive authentication screens using Thymeleaf & Tailwind CSS.
- Integrate standard password hashing with `Argon2PasswordEncoder`.

Phase 3: Security Hardening & Session Revocation

- Enforce Proof Key for Code Exchange (PKCE) for authorization requests.
- Integrate Redis for active session tracking and instant JWT/Refresh Token blacklisting.
- Add Time-based One-Time Password (TOTP) Multi-Factor Authentication.

Phase 4: Observability & Deployment

- Provide multi-stage containerization via `Dockerfile` and `docker-compose.yml` (App + Postgres + Redis).
- Configure Spring Boot Actuator with Prometheus metrics for authentication performance monitoring.
- Generate OpenAPI 3.0 documentation for the administrative API layer.

5. Important Learning Points & Key Takeaways

- **Standardized Protocols:** Mastered RFC 6749 (OAuth 2.0) and RFC 7636 (PKCE) specifications.
- **Asymmetric Cryptography:** Direct experience managing RSA key pairs, JWT signatures, and JSON Web Key Sets (JWKS).
- **Hybrid Session Security:** Combining stateless JWT validation with distributed Redis state for instant token revocation.
- **Deep Spring Security Customization:** Extending default filter chains and authentication providers without violating standard OAuth specs.

Resume Summary Example

Project: Clavis-IAM — Lightweight Distributed OAuth2/OIDC Identity Provider

- *Architected a multi-tenant Identity & Access Management system using Spring Boot 3, Security 6, and PostgreSQL, fully compliant with OIDC & OAuth2 (PKCE) standards.*
- *Designed RSA-signed JWT token issuance and dynamic JWKS endpoints (`/well-known/jwks.json`) enabling distributed stateless token verification.*
- *Integrated Redis for high-throughput session tracking and token revocation (JTI blacklisting) maintaining*

